Using NEAT to optimise parameters for coevolution game with Braitenberg vehicles

Temir Zhumagali 22512702

May 21, 2026

1 Abstract

This paper modulates coevolution game with Braitenberg vehicles using microbial genetic algorithm for changing controller parameters to simulate adaptation. Experiment displayed emergence of different behavioral categories, such as follower, chaotic and avoiding strategies. Different strategies performed differently depending on opponent. In order to try to create more universal strategy, performance of each behavior was tested against different opponent. Contrary to the idea, agents developed all-or-nothing strategy, which is referred to as launching by this paper.

2 Introduction

This project aims to expand the experiment conducted on previous assignment. The goal remains the same, to simulate dynamic system of Braitenberg's vehicles for coevolution game with hunters and runners. However previously vehicles had simple feedforward controllers and simple environment. Actuators were directly connected to sensors with weight, adjusted by microbial GA. This project introduces a new NEAT based controller and new environment. Braitenberg's vehicles are agent models producing sensory motor behavior (Braitenberg, 1984).

At the front of the vehicles sensors to perceive stimuli and at the sides two wheels to produce movement are placed. In this experiment 10 agents are divided into two groups: hunters and runners. The goal of the hunters is to catch the runners, while the goal for the runners is to reach the escape area that is located behind hunters. For this experiment additional win condition for runners have been added. Runners are required to touch green light sources before running away.

By binding sensors directly to wheel motors, those agents are able to produce complex behavior by following simple rules. Using a neural network instead of simple wiring introduces additional complexity to the system. However it is not known which NN structure will be the most effective for runners and hunters. To solve this issue NeuroEvolution of Augmenting Topologies (NEAT) method has been introduced (Stanley & Miikkulainen, 2002). The idea of NEAT is to use evolutionary algorithms to evolve the structure of NN alongside weight and biases. Stanley and Miikkulainen (2002) claim that this system outperforms fixed topology methods due to employing a principled method of crossover of different topologies, protecting structural innovation using speciation, and incrementally growing from minimal structure.

Coevolution is a process of mutual adaptation of two or more species to each other. Individuals from both of the populations take turns for being tested and testing competing population members. The fitness of one member of the population is directly affected by competition with members of the opposite population (Rosin & Belew, 1997). Success of one runner implies defeat of a hunter and vice versa. However, since one member of runner populations has multiple runners to simulate, defeat of one runner does not lead to defeat of population. Instead, fitness of a member of the runners population depends on the performance of all runners. The same applies to the population of hunters.

When the genotype of one member of the runner population leads to poor performance, it is possible for evolution to modify the controller in a way that would counter the reason for its failure. Increased performance of runners lead to decreased performance of hunters, which will lead to its evolution. Continuation of this process among species may lead to escalating adaptations to one another, referred to as the evolutionary "arms race" (Dawkins & Krebs, 1979).

Another coevolutionary strategy that is contrary to arms race, is strategy chases, introduced by (Wagner et al., 2013). If arms race describe escalating evolution, e.g. faster, stronger, more agile, strategy chases introduces idea of cyclic dominance, similar to rock-paper-scissors. This paper explores applicability of both strategies for the system.

Both of the runners and hunters possess the property of adaptability. Adaptation is commonly referred to as a process by which an organism alters its characteristics that affect survival or reproduction (Futuyma & Kirkpatrick, 2017). For this project adaptability will be defined as a property of a group of agents to modify their behavior with respect to each other in order to achieve the same goal, whether it is to escape from or catch an opponent.

3 Methodology

3.1 Braitenberg's vehicles

Braitenberg vehicles are modeled with Sandbox API (Johnson, 2024). Sandbox provides a variety of useful tools for modulating behavior, including creation and configuration of environment, agents, sensors, etc.

Sandbox simulates the system by steps. At the beginning the system is created according to set initial parameters. At each step the position of agents changes according to the right and left wheel speed that is calculated by the controller. Controller input is a vector of values from the sensors and output (speed of left and right wheel) is output of the NN model multiplied by 1.5 (because tanh activation function's outputs are between -1 and 1, which is too low as a speed). Value is the brightness of a light source that a sensor can detect. A sensor can detect light as the angle of vector from a light source if the absolute distance between vector angle and angular orientation of the sensor is less or equal to half of FOV of the sensor.

Sandbox is capable of simulating multiple models of light, including inverse square, linear and binary. For this project a linear model of light was chosen. The brightness perceived by the sensor is calculated using formula:

$$\text{brightness} = \max(\text{brightness}_{max} - \text{gradient} * \text{distance}, 0)$$

The distance is calculated as Euclidean norm:

$$\text{distance} = \sqrt{x_{light}^2 + x_{sensor}^2 + y_{light}^2 + y_{sensor}^2}$$

Each dynamic system changes its states with time (Hirsch & Smale, 1984). The states of the system changes according to the formulas:

$$v = \frac{v_{left} + v_{right}}{2}, \omega = \frac{v_{right} - v_{left}}{2r}$$

$$\mathbf{state} = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix}$$

$$\dot{\mathbf{state}} = \begin{bmatrix} v\cos(\theta) \\ v\sin(\theta) \\ \omega \end{bmatrix}$$

$$x_{t+1} = x_t + dt \cdot v\cos(\theta_t)$$

$$y_{t+1} = y_t + dt \cdot v\sin(\theta_t)$$

$$\theta_{t+1} = \theta_t + dt \cdot \omega$$

3.2 NEAT

The NEAT method is implemented using the neat-python library (McIntyre et al., 2026).

The genotype contains a list of connection genes, each contains information about one connection. It includes end-node of connection, in-node, weight of connection, whether connection is maintained, unique innovation number, which will be used in crossover (Stanley & Miikkulainen, 2002).

Mutation is the process of creating or deleting connections between nodes, nodes themselves with certain probability and modifying weights of connections. A new node is created in the middle of the existing edge, leading to old connections being disabled and creation of two new connections between new node and in-node and end-nodes of old connection. Major parameters of the simulation are presented in Table 1.

Parameters	**Values**
activation function	tanh
pop_size	3
node_add_prob	0.2
node_delete_prob	0.2
conn_add_prob	0.3
conn_delete_prob	0.2
weight_max_value	30
weight_min_value	-30
weight_mutate_power	0.5
weight_mutate_rate	0.8
weight_replace_rate	0.1

Table 1: Parameters of the simulation

In order to perform crossover, it is required to keep track of the innovation number, which represents the chronological order of creation of each gene. With innovation numbers the systems has information which genes match up. During crossover genes with the same innovation number line up, and when innovation numbers don't match gene is inherited from more fit parent and either disjoint or excess. Example given in Figure 1.

Usually after mutating new topology fitness of the model decreases. To address this issue the process of Protecting Innovation through Speciation is introduced. Individuals within the same population compete with each other to determine which member's genes will be passed. Historical markings (innovation number) allows the system to divide population into different

groups based on topology. It is possible to measure compatibility distance by using formula:

$$\delta = \frac{c_1 E}{N} + \frac{c_2 D}{N} + c_3 \overline{W}$$

where δ compatibility distance, E is number of excess, D is number of disjoint and $\overline{W}$ is number of matching genes.

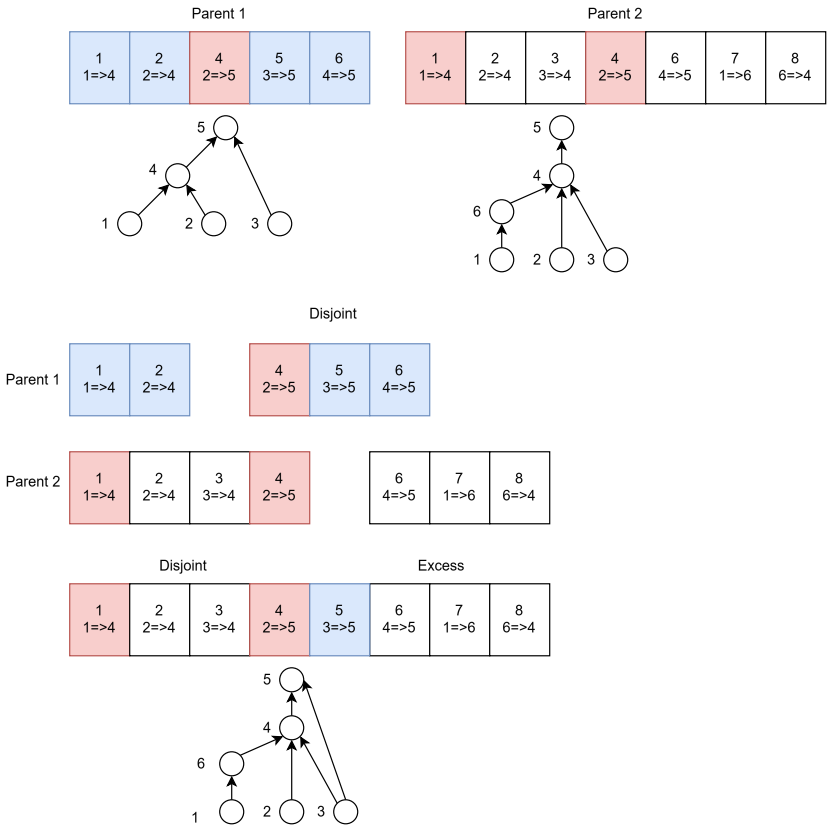


Figure 1: Matching up genomes for different network topologies using innovation numbers. Red squares represent disabled connections

NEAT also creates a threshold δ_t. During the testing, if the distance of a genome to a randomly chosen member of the population is less than said threshold, they are placed in the same group.

NEAT minimizes dimensionality by starting all genomes with zero hidden layers. Structural mutations introduce new structures and only structures that have been proven to be useful through fitness evaluations survive.

3.3 Fitness functions

In this project two types of formulas is realized. One considering distance from target and one does not. All fitness functions aims to introduce positive feedback loop to the system, encouraging agents.

Fitness for hunters without considering the distance:

$$\text{fitness}_{\text{hunter}} = \frac{2\left(\frac{\text{catched_runners}}{5}\right) + \left(1 - \frac{\text{time}}{11}\right)}{3}$$

3 maximum possible value, achieved from $2\left(\frac{\text{catched_hunters}}{5}\right) + \left(1 - \frac{\text{time}}{11}\right)$.

Fitness for runners without considering the distance:

$$\text{fitness}_{\text{runners}} = \frac{2\left(\frac{\text{won_runners}}{5}\right) + \left(1 - \frac{\text{time}}{11}\right)}{4}$$

Fitness for hunters considering the distance:

$$\text{fitness}_{\text{hunter}} = \frac{2\left(\frac{\text{catched_runners}}{5}\right) + \left(1 - \frac{\text{time}}{11}\right) + (1 - min(\frac{d}{50}, 1)}{4}$$

where d is smallest distance between hunter and runner at the end of simulation.

Fitness for runners considering the distance:

$$\text{fitness}_{\text{runners}} = \frac{2\left(\frac{\text{catched_runners}}{5}\right) + \left(1 - \frac{\text{time}}{11}\right) + (1 - min(\frac{h}{50}, 1) + \frac{lights}{5}}{5}$$

where h is smallest distance between runner and goal line, lights is number of lights that runners managed to touch.

4 Results and Analyses

4.1 Analyzing models without additional goal for runners

The first run of population for 400 generations generated hunters with two types of behaviors. The first one is simple circling at the starting position and the second can be described as sweeping.

Circling hunters appear to set the speeds of one of their wheels to 0 or very small numbers and keep the speeds of other wheel high, thus circling on the same spot. Swing hunters gazer into one group, keeping close distance with each other, move slowly towards one side of the arena, thus becoming



Figure 2: Examples of trajectories of sweeping and circling behaviors

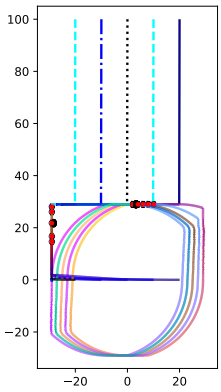
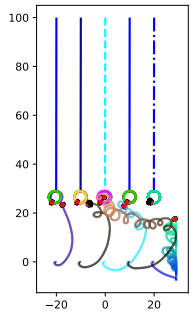


Figure 3: Sweepers creating circle with their trajectories

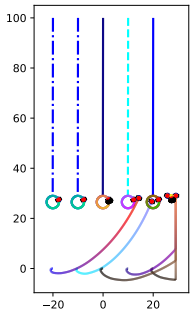
an obstacle for runners, then move up. Even if duration of simulation would increase, sweepers would not stop moving in tandem and eventually would form a circle with their trajectories.

The behavior produced by runners can generally be decided as avoiding. The majority has a probability of becoming chaotic or traveling in a relatively straight avoiding behavior. There is also a quick turner that at the beginning of simulation turned around quickly and traveled in a relatively straight line. Another type of runners is that moves all members of population in a straight line towards the left wall. In the Figure 4 two of the patterns of second type is represented. Both figures represent the behavior of the same runner.

From the trajectory, it can be seen that the movement of the right-most runner was disrupted by the second right-most runner.The right-most runner reached the wall of the arena, unable to avoid its neighbor. Because



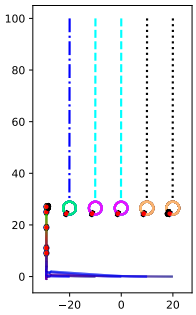
(a) Chaotic circular behavior



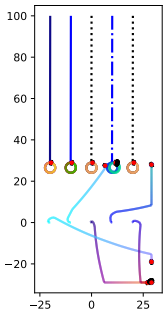
(b) Relatively straight avoiding behavior

Figure 4: Examples of trajectories of sweeping and circling behaviors

two runners were too close to each other one of them made drastic turn to the opposite side, now being too close to another runner and changing its directory. Testing the same runner does not always lead to this situation, as seen in Figure 4. Which type of behavior is produced depends on the speed of the right-most runner.



(a) Move one direction and slide down



(b) Quick turn and moving at straight line.

Figure 5: Behavior of the other runners

Behavior of runners depicted in Figure 5 has been recorded several times, confirming that it is not coincidence.

Assessing both runners and hunters, it is evident that groups produce coevolutionary predator-prey patterns, described by (Wagner et al., 2013): a) hunters that circle stationary can catch or divert most of the runners

with chaotic circular or straight avoidant behavior, and because they do not leave their orbit, those hunters can not catch runners that move towards the left wall and slide down, b) however, left going sweepers can easily catch those runners, as they search for enemies starting from left wall. c) Chaotic/avoidant runners can win against left going sweepers as those runners tend to move to the right side, however they can be caught by right going sweepers due to the same reason. Property that makes one agent strong against certain opponent at the same time makes it weak against another.

4.2 New environment

In this section new elements will be introduces to the environment and new corresponding fitness functions, described in the methodology section. A green lights are randomly placed in the environment and runners have to touch those lights before reaching the goal line. Otherwise they will not be able to escape. Runners and hunters both are given green light sensors to introduce additional complexity for both of them. Due to the time constrain this simulation was conducted only once for 200 generations with 3 population members.

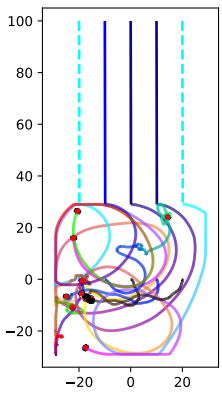


Figure 6: Runners and hunters trying to reach green light source

Hunters were able to be better at following the green lights because runners had to also run away from hunters. Some hunters and runners failed to produce any meaningful behavior, example given in Figure 7. Evidently population of 3 and 200 generations were not enough for proper development of the system.

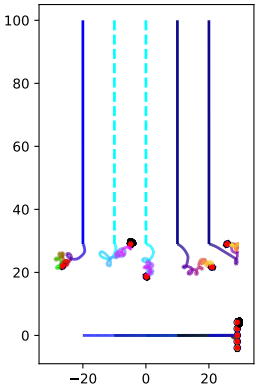


Figure 7: Runners and hunters trying to reach green light source

5 Discussion

The results demonstrate the development of adaptive predator-prey behavior. As described at the beginning, of the paper adaptability a property of a group of agents to modify their behavior with respect to each other in order to achieve the same goal, whether it is to escape from or catch an opponent. The results support this definition, because both runners and hunters managed to produce new behavior, for runners more suitable to escape from predators and for hunters to catch as much prey as possible. For example, stationary circling hunters could intercept chaotic runners, but were weak against runners that were moving along the left wall. In contrast, sweeping hunters were effective against wall-following runners but less effective against chaotic avoidance strategies. This type of adaptability is resembles towards strategy chase rather than arms race. Instead of evolving to outmaneuver opponent, agents evolved different strategies. This results is similar to ones obtained in previous assignment, suggesting that it was not coincidence, but rather natural flow of evolution for this particular task.

The implementation of the new environment with green light sources was not successful. Introducing an additional task increased the complexity of the environment and required runners to balance two competing objectives: collecting lights and avoiding hunters. Hunters also reacted to green lights because they possessed the same sensors. While some cooperative behavior evolved, results were mostly disappointing. That confirming thoughts that the population size and number of generations required to obtain partially satisfactory results from problem with increased complexity is higher than possible to simulate due to the time constrains. Neat does not have popula-

tion big enough to divide into groups based on the compatibility. It also does not have enough generations to develop sufficiently complex neural network, as NEAT starts with minimal complexity.

This project directly related to broader topics of adaptive systems, such as dynamic systems. The system not only has states and parameters that change according to rules described in methodology section, but the rules them self change each generation because of NEAT.

Future direction of research involve introducing noise sources and handling, evolving networks with recurrent connections and increasing size of the population.

References

Braitenberg, V. (1984). *Vehicles: Experiments in synthetic psychology*. MIT Press.

Dawkins, R., & Krebs, J. R. (1979). Arms races between and within species. *Proceedings of the Royal Society of London. Series B, Biological Sciences*, *205*(1161), 489–511. https://doi.org/10.1098/rspb.1979.0081

Futuyma, D. J., & Kirkpatrick, M. (2017). *Evolution* (4th ed.). Sinauer Associates.

Hirsch, M. W., & Smale, S. (1984). *Differential equations, dynamical systems, and linear algebra* (1st ed.). Academic Press.

Johnson, C. (2024). Sandbox [Course materials, Adaptive Systems, University of Sussex].

Rosin, C. D., & Belew, R. K. (1997). New methods for competitive coevolution. *Evolutionary Computation*, *5*(1), 1–29. https://doi.org/10.1162/evco.1997.5.1.1

Stanley, K. O., & Miikkulainen, R. (2002). Evolving neural networks through augmenting topologies. *Evolutionary Computation*, *10*(2), 99–127. https://doi.org/10.1162/106365602320169811

Wagner, G. P., Zaman, L., Dworkin, I., & Ofria, C. (2013). Behavioral strategy chases promote the evolution of prey intelligence. *arXiv preprint arXiv:1310.1369*. https://doi.org/10.48550/arXiv.1310.1369